Matemática Discreta 2

Aula 11 -Algoritmos em Grafos (parte 2): algoritmos de busca em grafos

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Recapitulando

Um grafo é

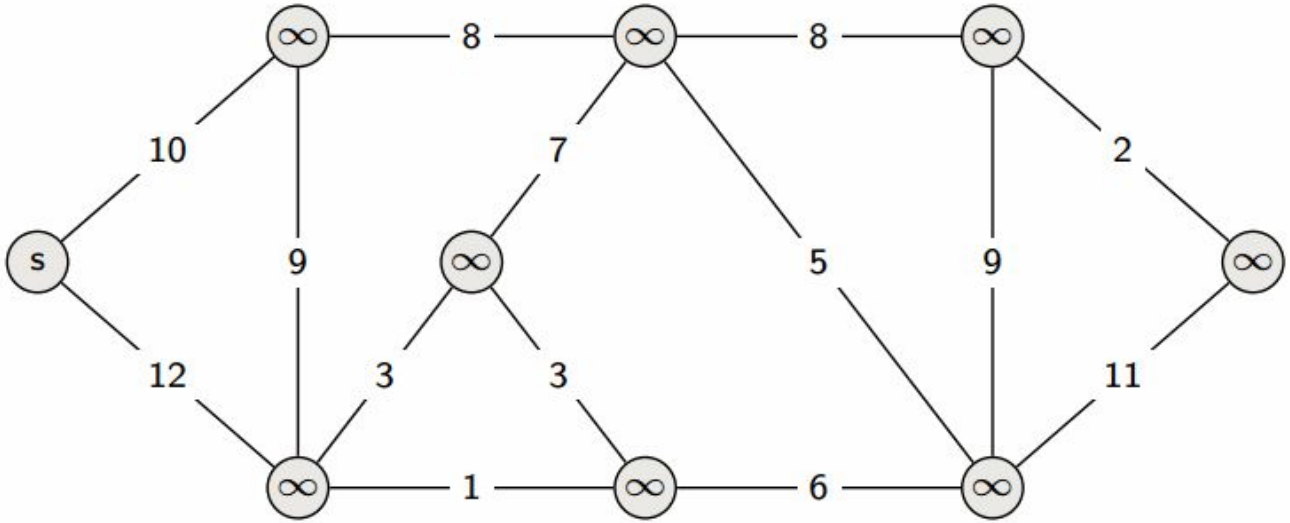
uma representação gráfica de

elementos de dados e

das conexões entre alguns destes itens

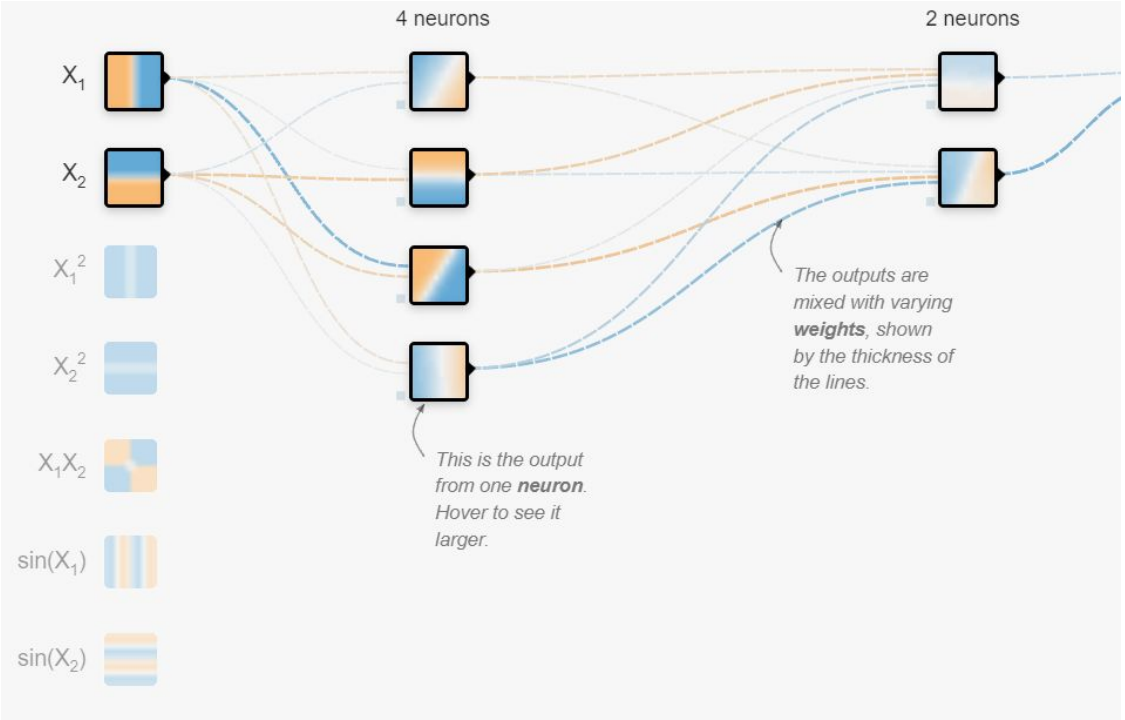
LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Aplicações: Waze



An Introduction to Graph Theory. Before diving into graph theory, we… | by Michelle Venables | Towards Data Science

Aplicações: Redes Neurais

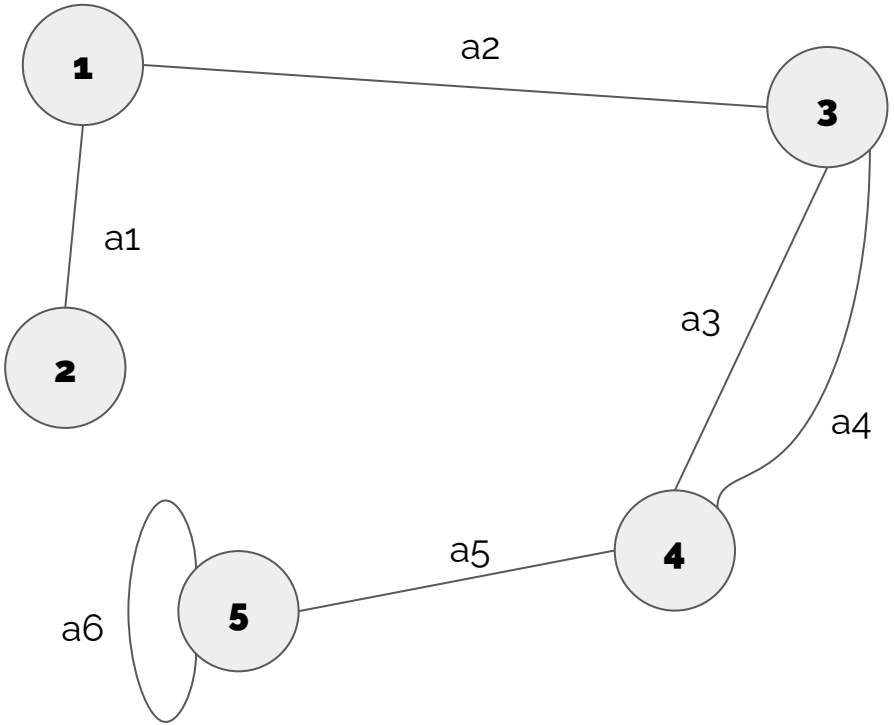


https://playground.tensorflow.org/

Trace um grafo

Que tenha os vértices {1, 2, 3, 4, 5}, as arestas {a1, a2, a3, a4, a5, a6} e a função

g(a1) = 1 -2,
g(a2) = 1-3,
g(a3) = 3-4,
g(a4) = 3-4,
g(a5) = 4-5 e
g(a6) = 5-5.

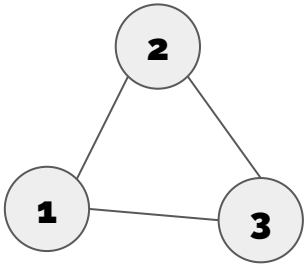


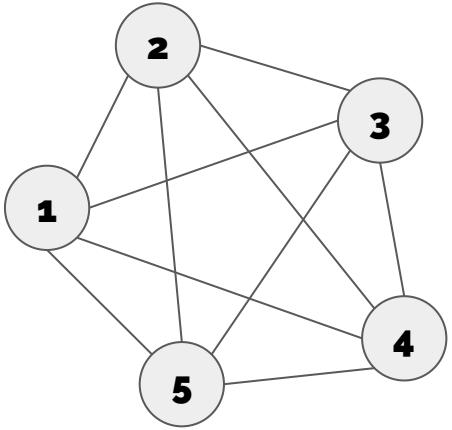
Grafo Completo

Um grafo completo é aquele no qual **todos os vértices distintos são adjacentes**

Geralmente é chamado de ***Kn***

Como podemos criar um grafo completo com 3 nós, e com 5?





Grafo Bipartido Completo

Grafos bipartidos completos são grafos em que os **vértices podem ser divididos em dois conjuntos disjuntos**, de tal forma que quaisquer dois vértices tomados no mesmo conjunto não são adjacentes, mas **todos dois vértices escolhidos em conjuntos diferentes são adjacentes**.

Geralmente chamados de ***Kn,m***

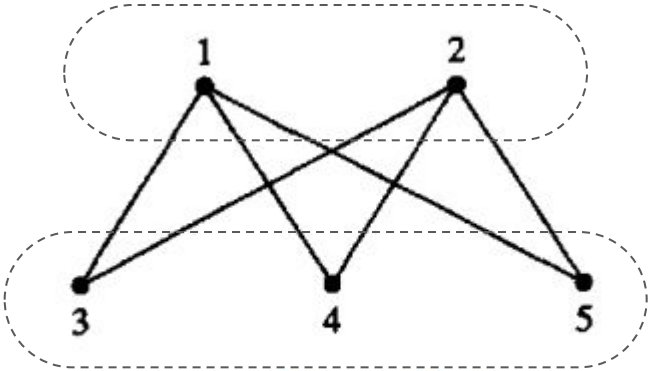
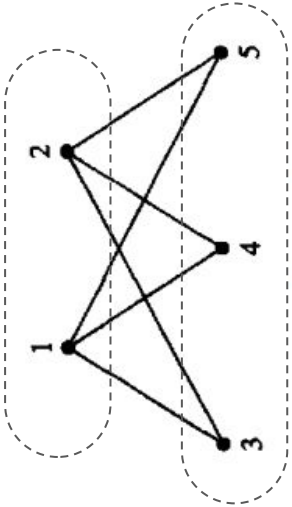


Figura 5.13



Árvores

Uma árvore é

um **grafo acíclico** e **conexo**

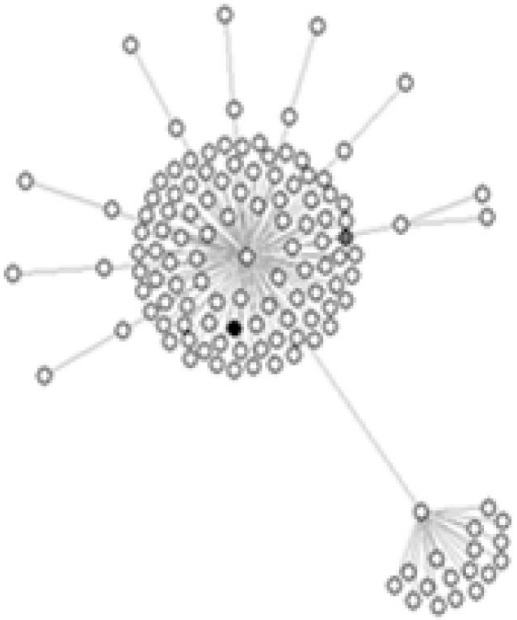
com um nó designado como a

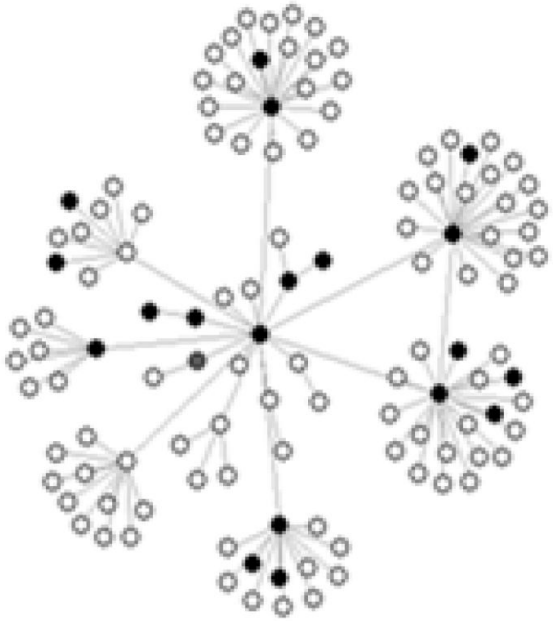
raiz da árvore

Como seria a representação visual de retweets?

Mima Diaz de Conduru 🚦🎀🔗 on Twitter: "Eu amo Bode Gaiato https://t.co/GdnXAbIAvn" / Twitter

Propagação de “Real vs Fake News” em retweets





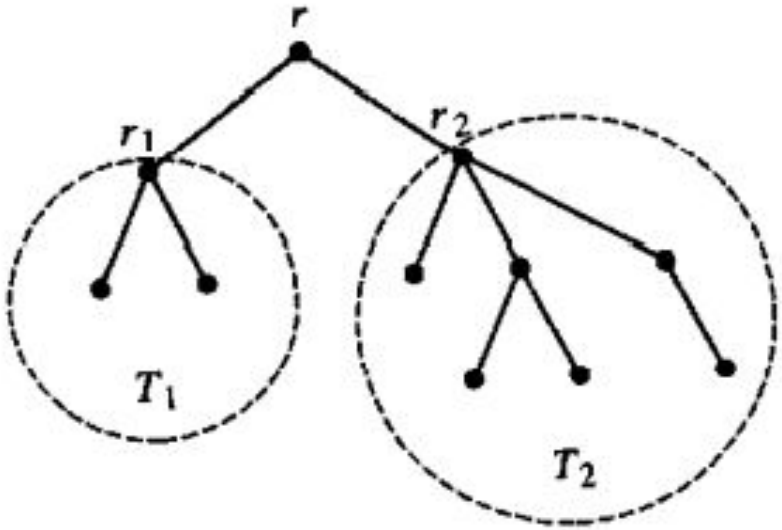
(a) TW propagation tree of fake news;

(b) TW propagation tree of real news

Electronics | Free Full-Text | Fake News Analysis Modeling Using Quote Retweet | HTML (mdpi.com)

Raiz, Vértices Pais e Filhos

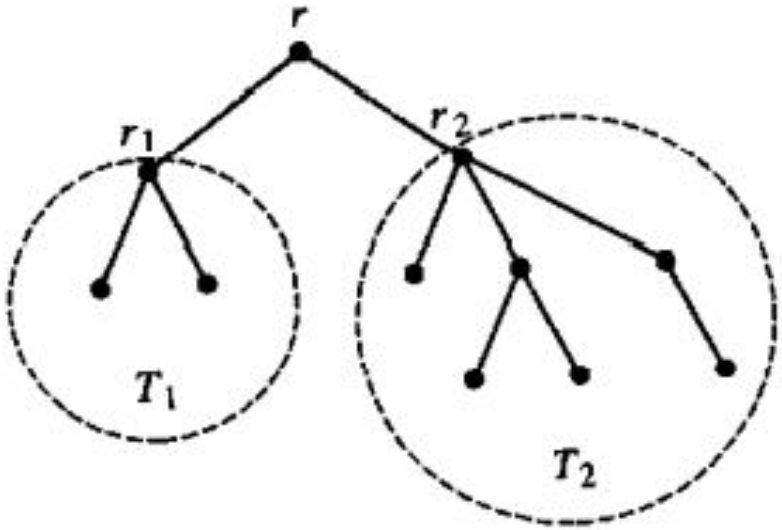
- **r** é a raiz da árvore ao lado
- os vértices **r1** e **r2** são filhos de **r**, e
- **r** é pai de **r1** e **r2**
- **r1** e seus filhos podem ser considerados uma sub-árvore **T1**
- o mesmo vale para **T2**
- os nós que não têm filhos são chamados de **folhas**



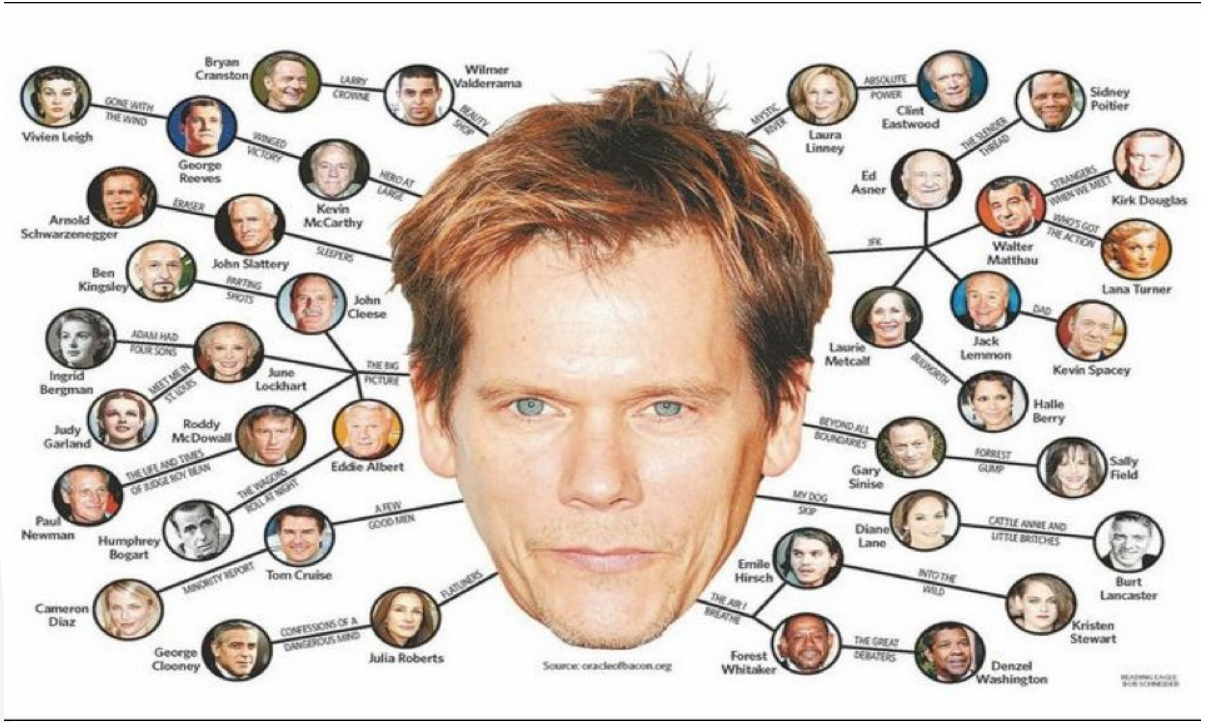
Profundidade de um Vértice

A profundidade de um vértice
em uma árvore é
o comprimento do caminho
da raiz até o vértice;

em particular,
a raiz tem profundidade **0**



Profundidade de um Vértice



Selton Mello has a Bacon number of 3.

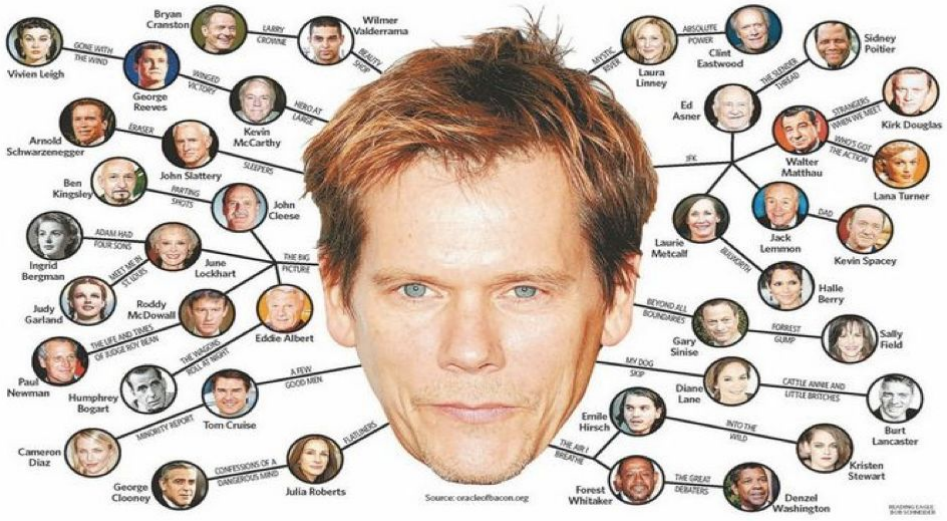
Selton Mello
was in
A Dog's Will
with
Matheus Nachtergaele
was in
Journey to the End of the Night
with
Brendan Fraser
was in
The Air I Breathe
with
Kevin Bacon

https://oracleofbacon.org/

Altura da Árvore

A altura (profundidade) da árvore é a maior profundidade de todos seus vértices; em outras palavras, é o comprimento do maior caminho entre a raiz e um vértice.

Qual é a altura/profundidade da árvore de Kevin Bacon?



Como buscar se um IG já está cadastrado?

@kevin

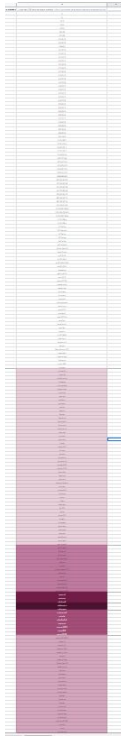
```
usuarios = {IG1, IG2, ..., IGn};
contaDisponivel = true;

for(Usuario usuario : usuarios) {
        if(usuario == @kevin) {
                print("IG não disponível");
                contaDisponilvel = false;
        }
}
```

|usuarios| = n, para descobrir que o login está disponível temos que realizar **n operações**

Como buscar se uma senha já está cadastrada?

NordPass | Features | Pricing | Apps | Business | Blog | Help | Login

Position	Password	Number of users	Time to crack it	Times exposed
1. ↑ (2)	123456	2,543,285	Less than a second	23,597,311
2. ↑ (3)	123456789	961,435	Less than a second	7,870,694
3. (new)	picture1	371,612	3 Hours	11,190
4. ↑ (5)	password	360,467	Less than a second	3,759,315
5. ↑ (6)	12345678	322,187	Less than a second	2,944,615
6. ↑ (17)	111111	230,507	Less than a second	3,124,368
7. ↑ (18)	123123	189,327	Less than a second	2,238,694
8. ↓ (1)	12345	188,268	Less than a second	2,389,787
9. ↑ (11)	1234567890	171,724	Less than a second	2,264,884
10. (new)	senha	167,728	10 Seconds	8,213

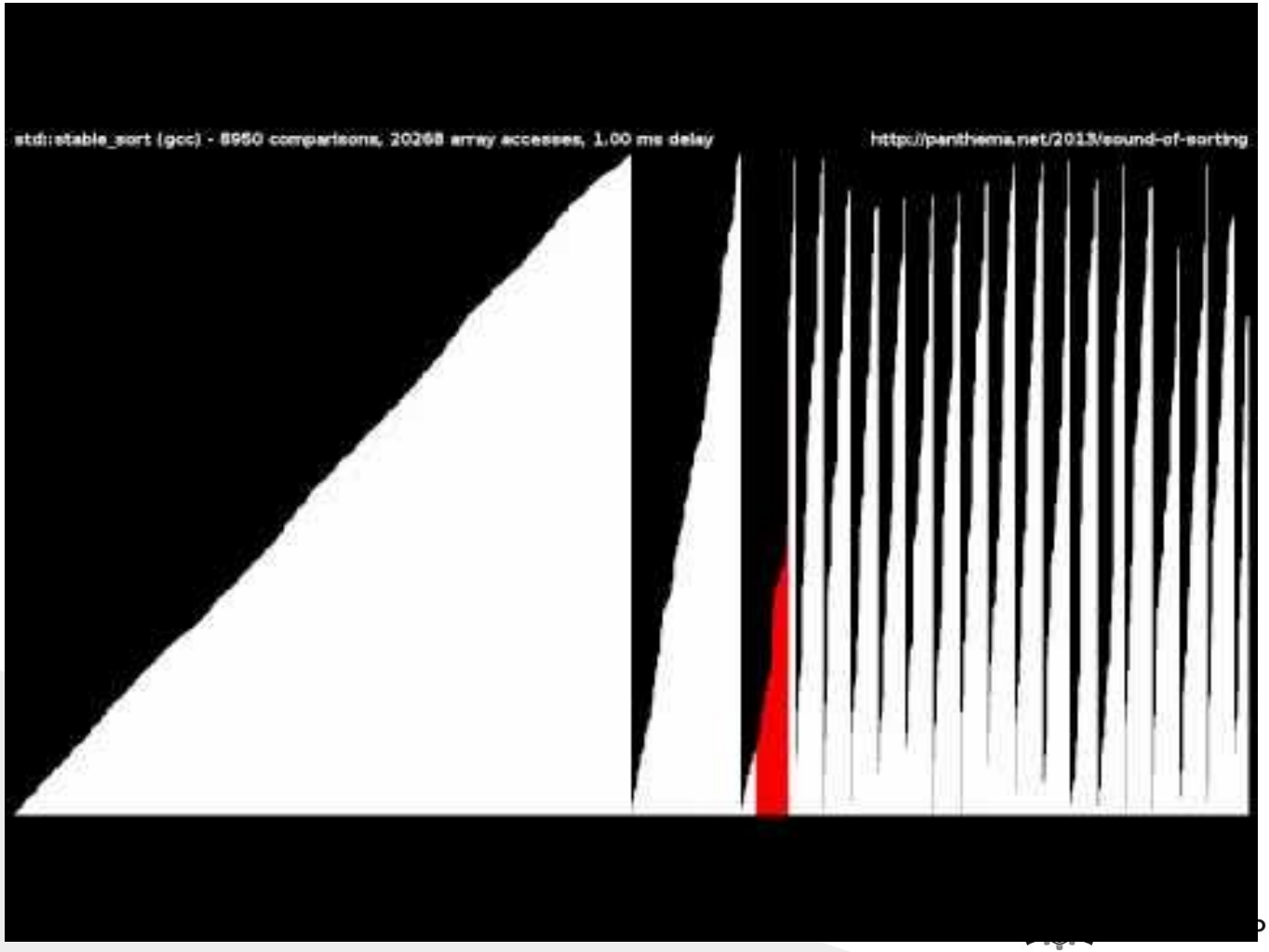


Lista das 200 senhas mais usadas:
https://nordpass.com/most-common-passwords-list

0
0
123
1111
1234
10203
12345
101010
102030
111111
112233
121212
123123
123321
123456
123456
123654
131313
142536
147258
159357
159753
222222
333333
456789

Top 200 Most Common Passwords of 2020 | NordPass

E se tivéssemos um conjunto de dados ordenado?



Busque o número 9 no array a seguir

| 2 | 5 | 8 | 9 | 10 | 12 | 14 |

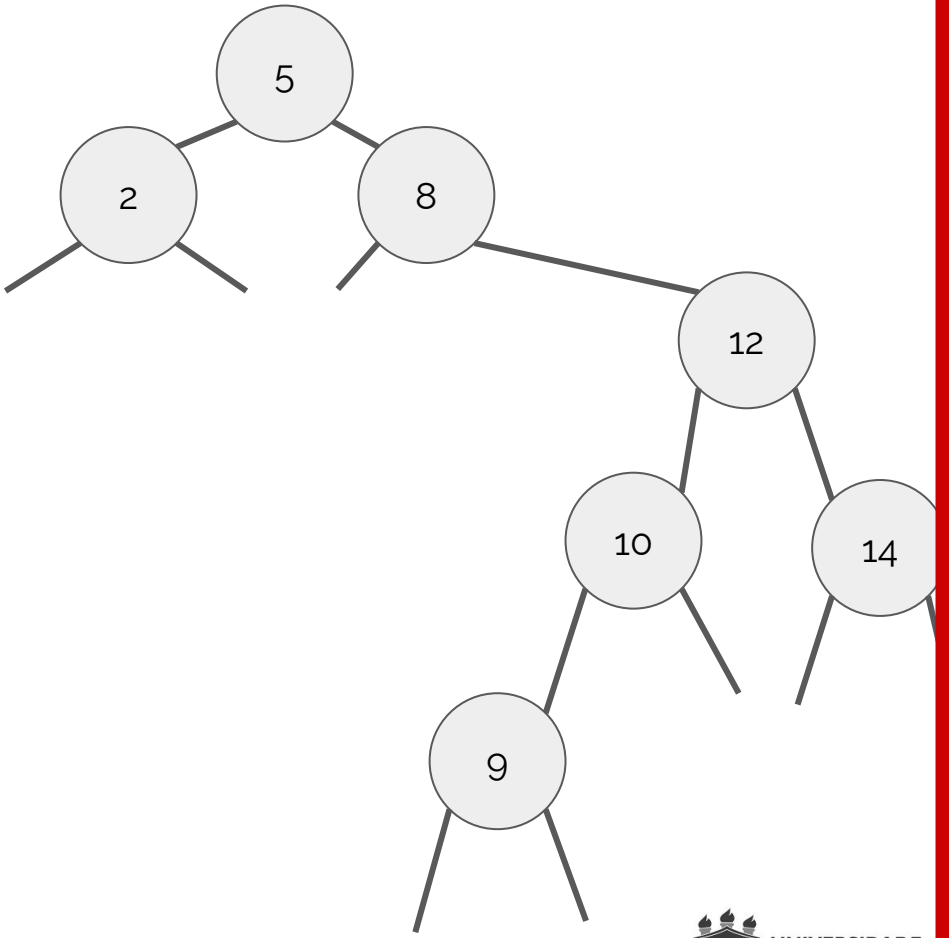
Árvores Binárias

São árvores em que cada nó tem **no máximo dois filhos**

Árvores Binárias: Construção

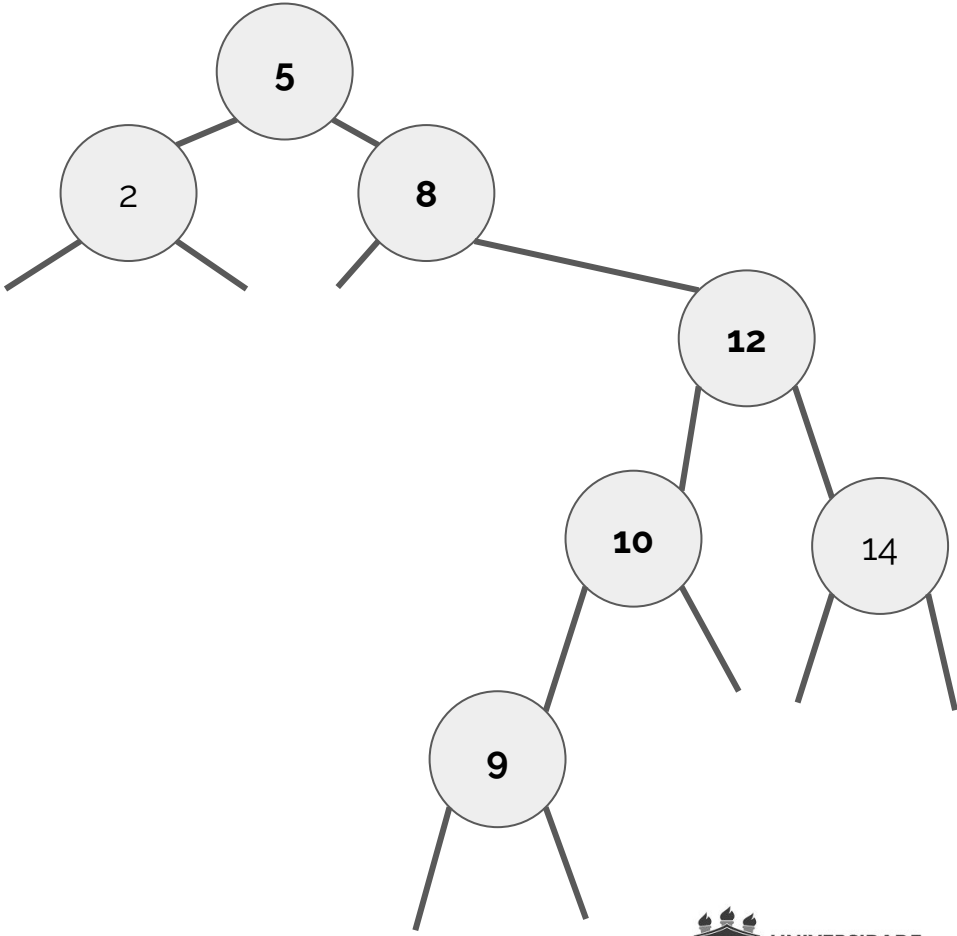
Crie uma árvore binária adicionando os elementos a seguir em ordem

5
8
2
12
10
14
9



Árvores Binárias: Prática

Busque o **número 9** na árvore criada



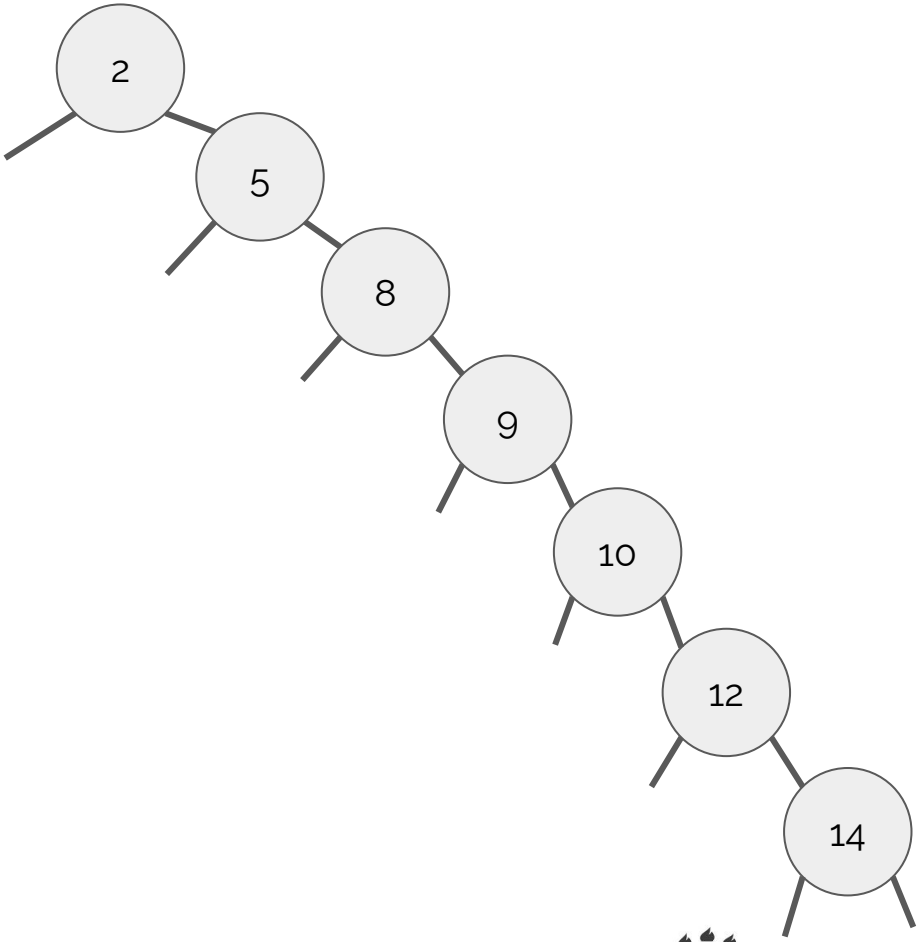
LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Árvores Binárias: Construção

Crie uma árvore binária adicionando os elementos a seguir em ordem

2
5
8
9
10
12
14

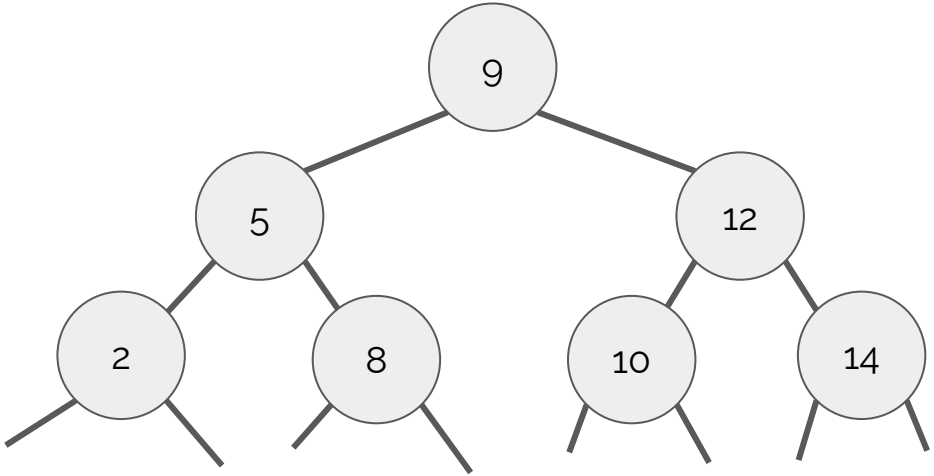
O que aconteceu com a árvore? A busca ainda é mais rápida nesta árvore?



Árvores Binárias: Construção

Crie uma árvore binária mais balanceada possível com os elementos a seguir

2
5
8
9
10
12
14



Árvores Binárias: Balanceamento

É possível reconfigurar uma árvore para balancear sua distribuição

Conceito de Hoje

Conceito de Hoje

Algoritmos em Grafos (parte 2)

Algoritmo de Dijkstra

LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

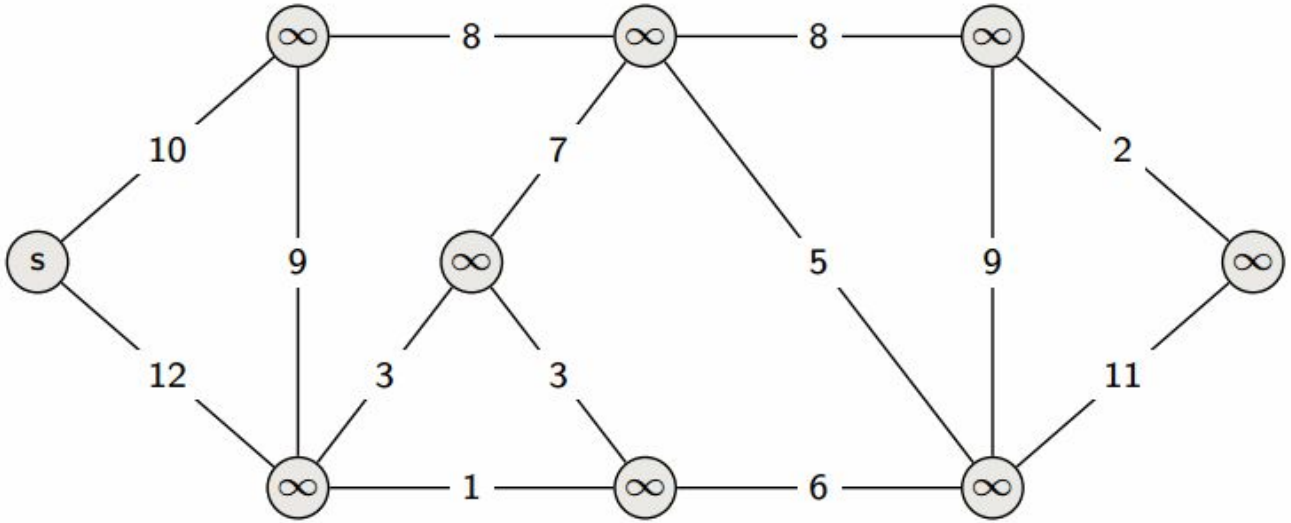
Introdução

Problema

Motivação

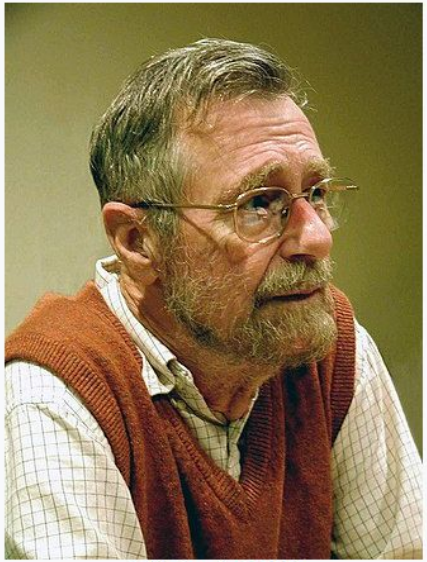
História e Pessoas

Algoritmo de Dijkstra



[An Introduction to Graph Theory. Before diving into graph theory, we… | by Michelle Venables | Towards Data Science]

Edsger W. Dijkstra



Dijkstra in 2002

Born	11 May 1930 Rotterdam, Netherlands

Apresentação

Definições

Exemplos Ilustrativos

1. **Marque todos os nós não visitados**. Crie um conjunto de todos os nós não visitados, denominado conjunto não visitado.

2. **Atribua a cada nó um valor de distância provisório**: defina-o como zero para nosso nó inicial e infinito para todos os outros nós.

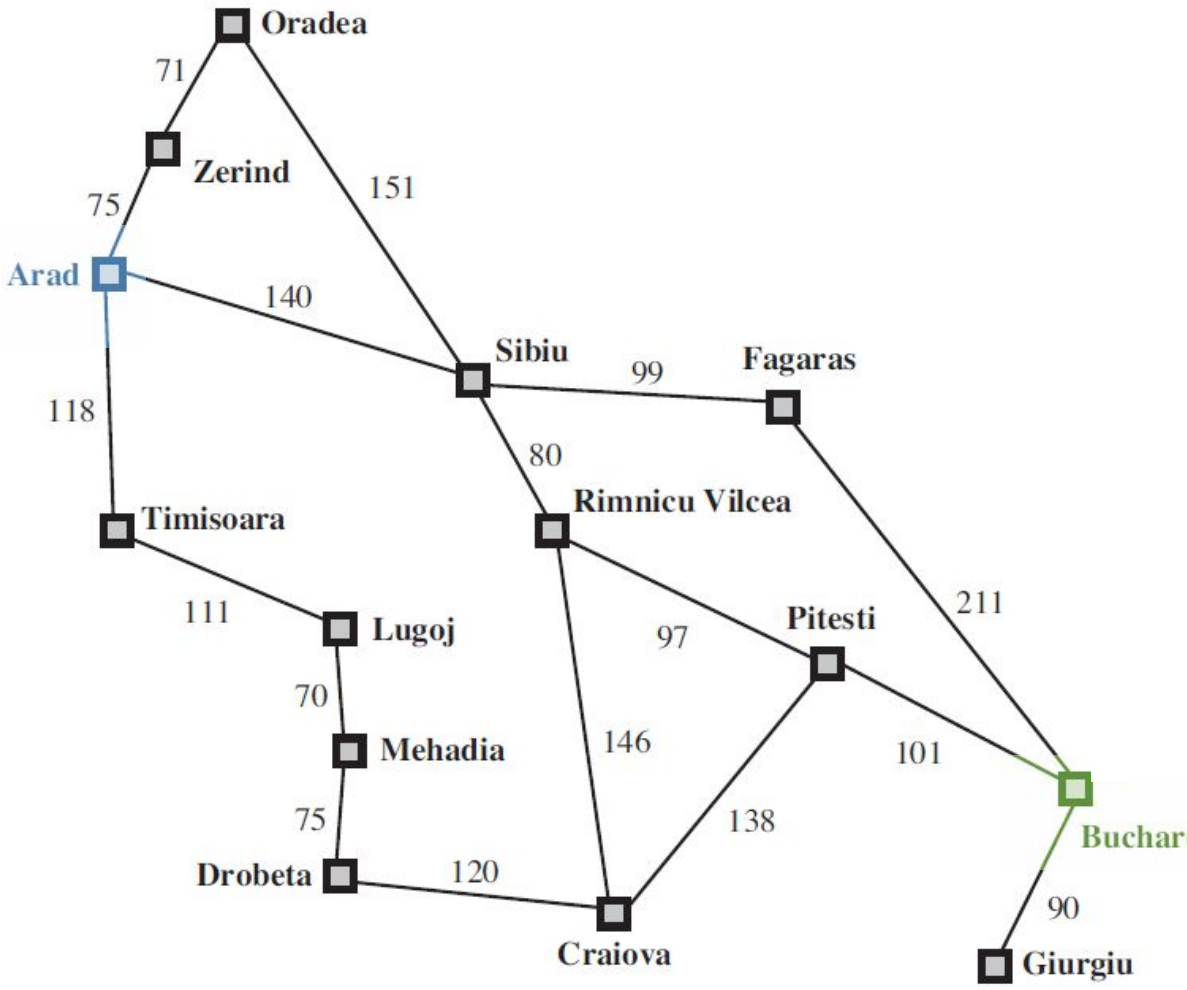
3. A distância provisória de um nó **v** é o comprimento do caminho mais curto descoberto até agora entre o nó **v** e o nó **inicial**. Visto que inicialmente nenhum caminho é conhecido para qualquer outro vértice além da própria fonte (que é um caminho de comprimento zero), **todas as outras distâncias provisórias são inicialmente definidas para o infinito**. Defina o nó **inicial** como **atual**.

4. Para o nó **atual**, considere todos os seus **vizinhos não visitados** e calcule suas distâncias provisórias até o nó atual. Compare a distância provisória recém-calculada com o valor atribuído anteriormente e **atribua o menor**.
Por exemplo, se o nó atual A está marcado com uma distância de 6, e a aresta que o conecta a um vizinho B tem comprimento 2, então a distância de B a A será 6 + 2 = 8. Se B foi previamente marcado com uma distância maior que 8, então mude para 8. Caso contrário, o valor atual será mantido.

5. Quando terminarmos de considerar todos os **vizinhos não visitados** do nó **atual**, marque o nó atual como **visitado** e remova-o do conjunto não visitado. Um nó visitado nunca será verificado novamente. Se o nó de **destino** foi marcado como visitado (ao planejar uma rota entre dois nós específicos) ou se a menor distância provisória entre os nós no conjunto não visitado é infinita (ao planejar uma travessia completa; ocorre quando não há conexão entre o nó inicial e os nós não visitados restantes) pare. O algoritmo terminou.

6. Caso contrário, selecione o nó não visitado que está marcado com a menor distância provisória, defina-o como o novo nó atual e volte para a etapa 3.

```
function Dijkstra(Graph, source):

    create vertex set Q

    for each vertex v in Graph:
        dist[v] ← INFINITY
        prev[v] ← UNDEFINED
        add v to Q
    dist[source] ← 0

    while Q is not empty:
        u ← vertex in Q with min dist[u]

        remove u from Q

        for each neighbor v of u still in Q:
            alt ← dist[u] + length(u, v)
            if alt < dist[v]:
                dist[v] ← alt
                prev[v] ← u

    return dist[], prev[]
```

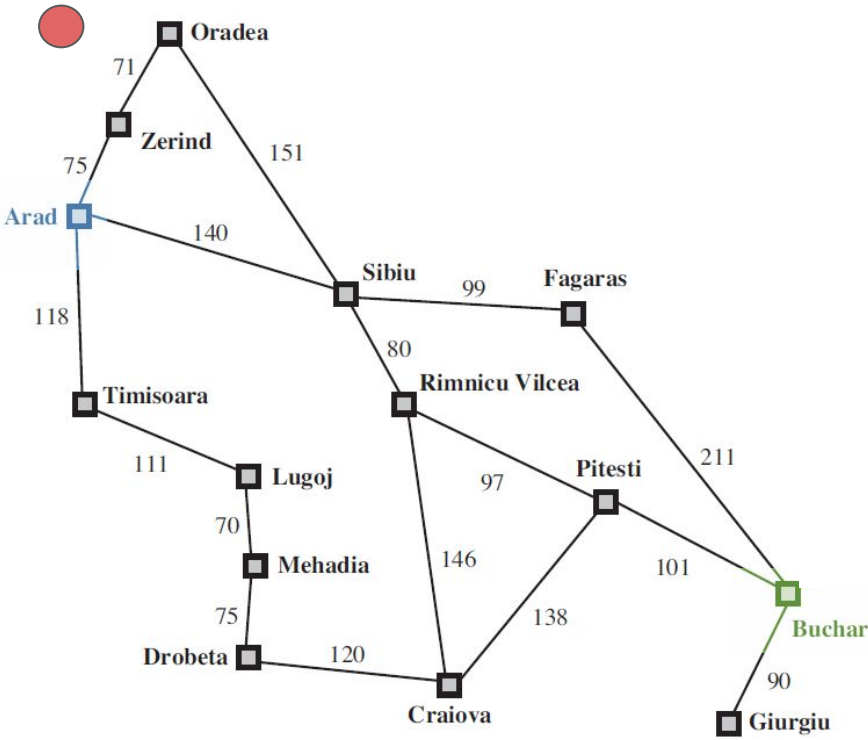


FIGUEIREDO DE PERNAMBUCO

```
function Dijkstra(Graph, source):

    create vertex set Q

    for each vertex v in Graph:
        dist[v] ← INFINITY
        prev[v] ← UNDEFINED
        add v to Q
    dist[source] ← 0

    while Q is not empty:
        u ← vertex in Q with min dist[u]

        remove u from Q

        for each neighbor v of u still in Q:
            alt ← dist[u] + length(u, v)
            if alt < dist[v]:
                dist[v] ← alt
                prev[v] ← u

    return dist[], prev[]
```

Dijkstra(Romania, Arad)

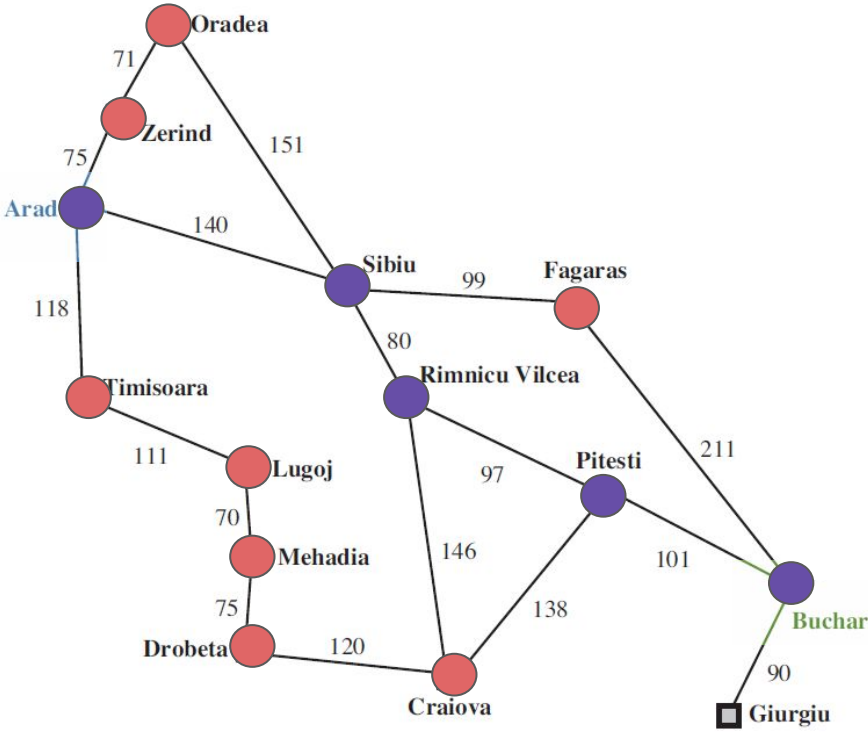
Q =	dist =	prev =
{	[	[
	A =	A = UN
}	Z = ∞	Z = UN
	O = ∞	O = UN
u=	S = ∞	S = UN
v=	T = ∞	T = UN
	L = ∞	L = UN
	M = ∞	M = UN
	D = ∞	D = UN
	C = ∞	C = UN
	R = ∞	R = UN
	F = ∞	F = UN
	P = ∞	P = UN
	B = ∞	B = UN
	G = ∞	G = UN
	]	]
	dist[u] = 366	
	length = 120	
	alt=486	



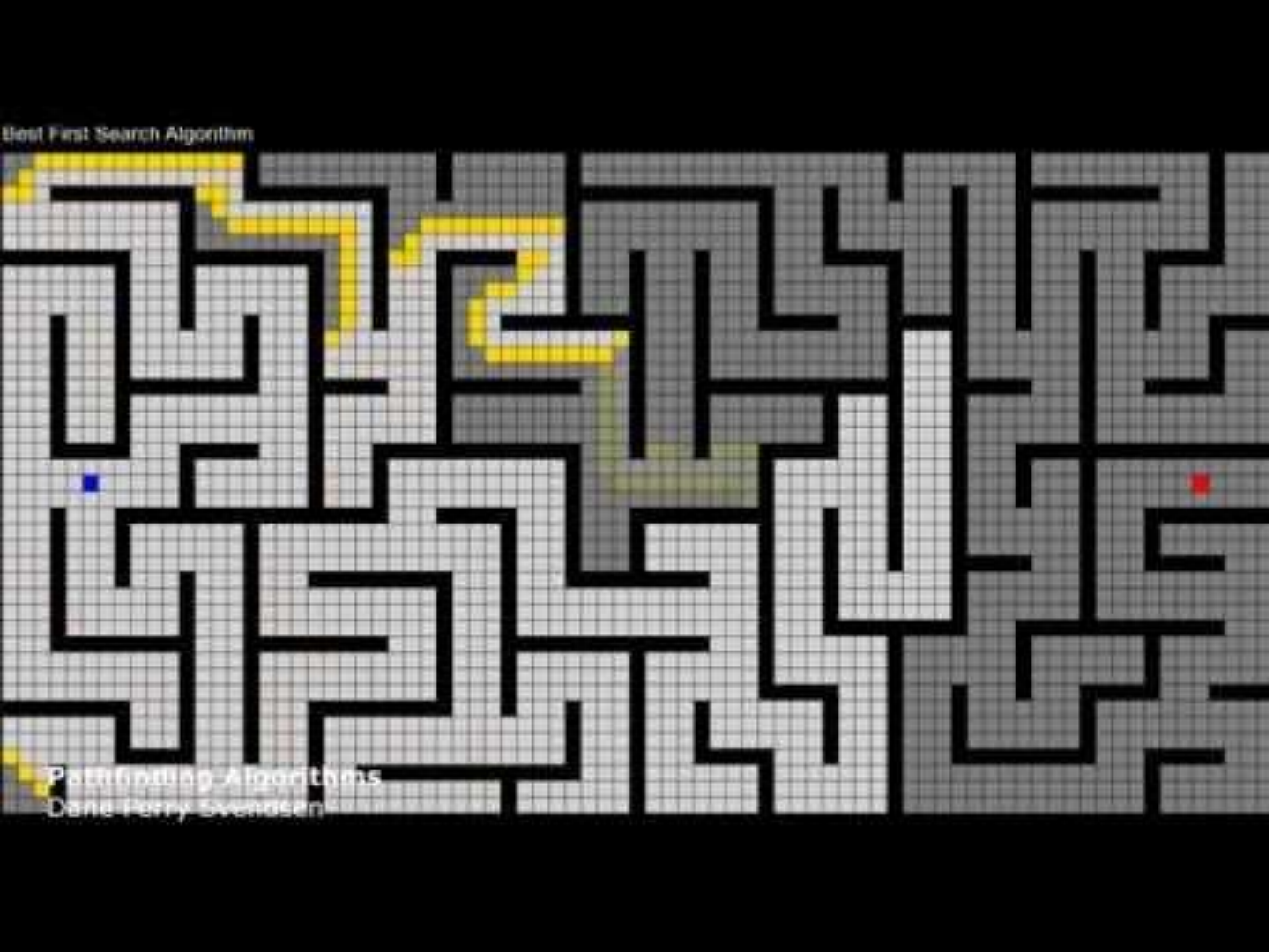
LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

```
function Dijkstra(Graph, source):

    create vertex set Q

    for each vertex v in Graph:
        dist[v] ← INFINITY
        prev[v] ← UNDEFINED
        add v to Q
    dist[source] ← 0

    while Q is not empty:
        u ← vertex in Q with min dist[u]

        remove u from Q

        for each neighbor v of u still in Q:
            alt ← dist[u] + length(u, v)
            if alt < dist[v]:
                dist[v] ← alt
                prev[v] ← u

    return dist[], prev[]
```

Q =	dist =	prev =
{	[	[
G	~~A = 0~~	A = ND
}	~~Z = 75~~	Z = A
	~~O = 146~~	O = Z
	~~S = 140~~	**S = A**
	~~F = 239~~	F = S
	~~T = 118~~	T = A
	~~L = 229~~	L = T
	~~M = 299~~	M = L
	~~D = 374~~	D = M
	~~C = 366~~	C = R
	~~R = 220~~	**R = S**
	~~P = 317~~	**P = R**
	B = 418	**B = P**
	G = ∞	G = ND
	]	]
u=B	**dist**[u]	
v=G	= 366	
	length	
	= 120	
	alt=486	



LUCAS SILVA FIGUEIREDO
UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO



LUCAS SILVA FIGUEIREDO

UNIVERSIDADE FEDERAL RURAL DE PERNAMBUCO

Livros > *Slides*

Universities should ban PowerPoint
https://www.businessinsider.com/universities-should-ban-powerpoint-it-makes-students-stupid-and-professors-boring-2015-6

Craig, Russell J., and Joel H. Amernic. "PowerPoint presentation technology and the dynamics of teaching." Innovative Higher Education 31.3 (2006): 147-160.

Referência

Fundamentos Matemáticos para Ciência da Computação (3a Edição)

6. Grafos e Árvores

   Seção 6.4: Terminologia e Aplicações de Grafos

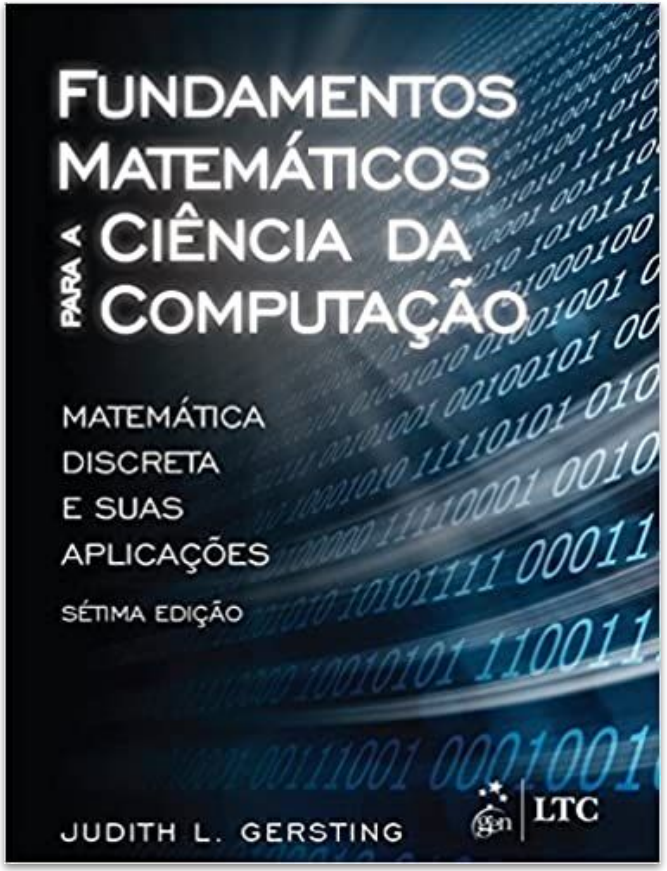
Algoritmo de Dijkstra

www.ime.usp.br/~pf/algoritmos_para_grafos/

Atualizado em 2020-03-04

Paulo Feofiloff

IME-USP

Algoritmo de Dijkstra

https://pt.wikipedia.org/wiki/Algoritmo_de_Dijkstra